1. Problem and Solution

Problem: A group of lizards live in a sago palm, and eat in the monkey grass across a driveway. Two cats sleep in the driveway, but wake up periodically. If a cat sees more than 4 lizards crossing, it plays with them, and the program ends. The program creates 20 lizard threads and 2 cat threads. Each lizard repeatedly sleeps, crosses to the monkey grass to eat, and then crosses back and sleeps again. The crossing must be synchronized so that no more than 4 lizards are on it at once, while still allowing up to 4 to cross simultaneously.

Solution: The drivewaySlots semaphore starts with four tickets, one per driveway spot. A lizard waits in sem_wait if all tickets are taken and returns its ticket with sem_post after crossing. The crossingMutex protects both crossing counts while lizards update them and cats read them.

2. Changes to Code

- Added drivewaySlots for available tickets and crossingMutex for the counts.
- In *lizardThread*, the loop calls *sleepNow*, *sago2MonkeyGrassIsSafe*, *crossSago2MonkeyGrass*, *madeIt2MonkeyGrass*, *eat*, *monkeyGrass2SagoIsSafe,* *crossMonkeyGrass2Sago*, and *madeIt2Sago*, in that order.
- Both IsSafe functions wait for a ticket. Both MadeIt functions return it.
- Both Cross functions update the counts under crossingMutex.
- Added runningMutex so cats and lizards read the same stop flag that main updates at shutdown. Main releases the lock before joining the threads.
- In main(), it now initializes everything, as well as deletes everything, to prevent memory leaks.

3. Results

WORLDEND	Lizards	Cats	Max Number of Lizards Crossing	Lizards Safe?
30	20	2	4	Yes
60	20	2	4	Yes
180	20	2	4	Yes
30	10	1	4	Yes
30	30	3	4	Yes

Results from the original test runs.

4. Issues Encountered

One issue was memory leaks. The cats and lizards were being properly deleted, but the cat and lizard threads were not, so it was fixed by adding destructors to lizard and cat classes.